Let's create the mapping for an index of `cafes` with a couple of fields. One of them is the `address` field, which points to the location of a cafe, represented as a `geo_shape` type.

Listing 12.4 Creating a mapping with the `geo_shape` field type

```
PUT cafes
{
  "mappings": {
    "properties": {
      "name":{
        "type": "text"
      },
      "address": {
        "type": "geo_shape"
      }
    }
  }
}
```

Sets the address type as geo_shape

The code creates an index called `cafes` to house local restaurants. The notable field is `address`, defined as a `geo_shape` type. This type now expects inputs of shapes in GeoJSON or WKT. For example, to represent a point on a map, we can input the field using `Point` in GeoJSON or `POINT` in WKT.

Listing 12.5 Inputting a `geo_shape` using WKT and GeoJSON formats

```
PUT cafes/_doc/1
{
  "name":"Costa Coffee",
  "address" : {
    "type" : "Point",
    "coordinates" : [0.17, 51.57]
  }
}

PUT /cafes/_doc/2
{
  "address" : "POINT (0.17 51.57)"
}
```

Inputs the address in GeoJSON format

Sets the address type as geo_shape

Coordinates (longitude and latitude) representing the Point

Inputs the address in WKT format

Sets the address type as POINT in WKT format

This code declares two ways to input a `geo_shape` field: GeoJSON and WKT. GeoJSON expects a `type` attribute of an appropriate shape (`"type":"Point"`) and the corresponding coordinates (`"coordinates":[0.17, 51.57]`), as in the first example. The second example in listing 12.5 shows the mechanics of creating a point using a WKT format (`"address": "POINT (0.17 51.57)"`).

NOTE There is a subtle difference when representing coordinates using a string format versus other formats. The string format expects the values in the order latitude followed by longitude, separated by a comma: for example,